



**RAJALAKSHMI ENGINEERING COLLEGE,
CHENNAI**
Department of Artificial Intelligence & Data Science
CS23633 – Cloud Computing Laboratory

Experiment 2

PAAS in Public Cloud Using Render

AIM

To build a basic Python Flask web application and deploy it on the Render cloud platform. This experiment helps in understanding how a Platform as a Service (PaaS) allows applications to run online without manually setting up or maintaining servers.

What is PaaS?

Platform as a Service (PaaS) is a cloud service that provides everything needed to develop, deploy, and run an application. It takes care of server management, software installation, and deployment, allowing developers to focus only on writing the application.

Why Render?

Render is an easy-to-use cloud hosting platform that supports web application deployment directly from GitHub. It automatically builds and hosts the application, making deployment simple even for beginners.

PROCEDURE:

Step 1

Sign in to your GitHub account or create one if you do not already have an account.

Step 2

Create a project folder named **RenderHelloApp**.

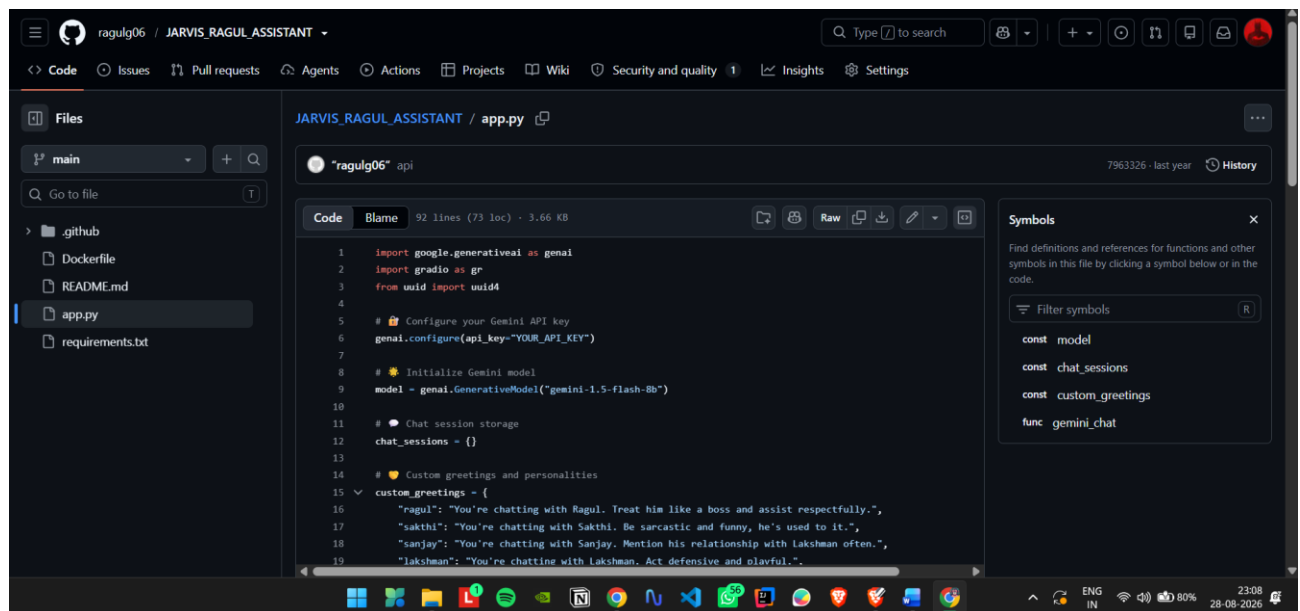
Step 3

Inside the project folder, create the following files:

- app.py
- requirements.txt

Step 4

Write the Flask application code in the app.py file.



```
1 import google.generativeai as genai
2 import gradio as gr
3 from uuid import uuid4
4
5 # 🗝️ Configure your Gemini API key
6 genai.configure(api_key="YOUR_API_KEY")
7
8 # 🌟 Initialize Gemini model
9 model = genai.GenerativeModel("gemini-1.5-Flash-8b")
10
11 # 💬 Chat session storage
12 chat_sessions = {}
13
14 # 🗨️ Custom greetings and personalities
15 custom_greetings = {
16     "ragul": "You're chatting with Ragul. Treat him like a boss and assist respectfully.",
17     "sakthi": "You're chatting with Sakthi. Be sarcastic and funny, he's used to it.",
18     "sanjay": "You're chatting with Sanjay. Mention his relationship with Lakshman often.",
19     "lakshman": "You're chatting with Lakshman. Act defensive and playful."
20 }
```

Step 5

Add the required Python packages such as **Flask** and **Gunicorn** to the requirements.txt file.

Step 6

Create a new public repository called **RenderHelloApp** on GitHub.

Step 7

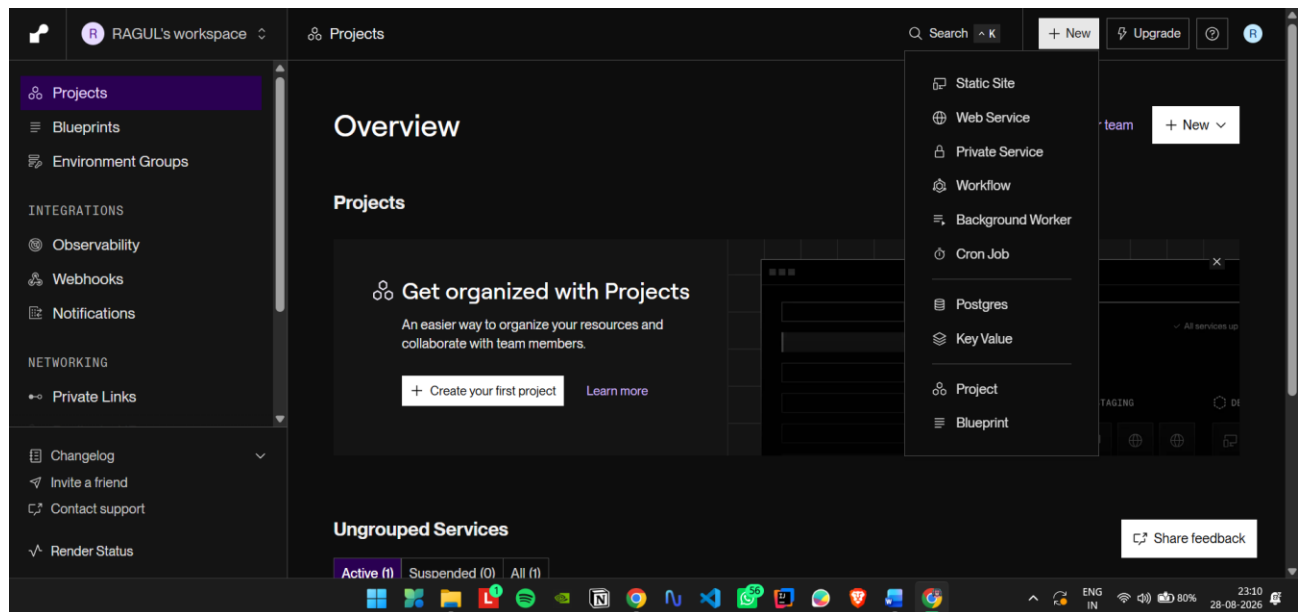
Upload both project files to the repository and commit the changes.

Step 8

Create an account on Render and connect it to your GitHub account.

Step 9

Create a new Web Service in Render.



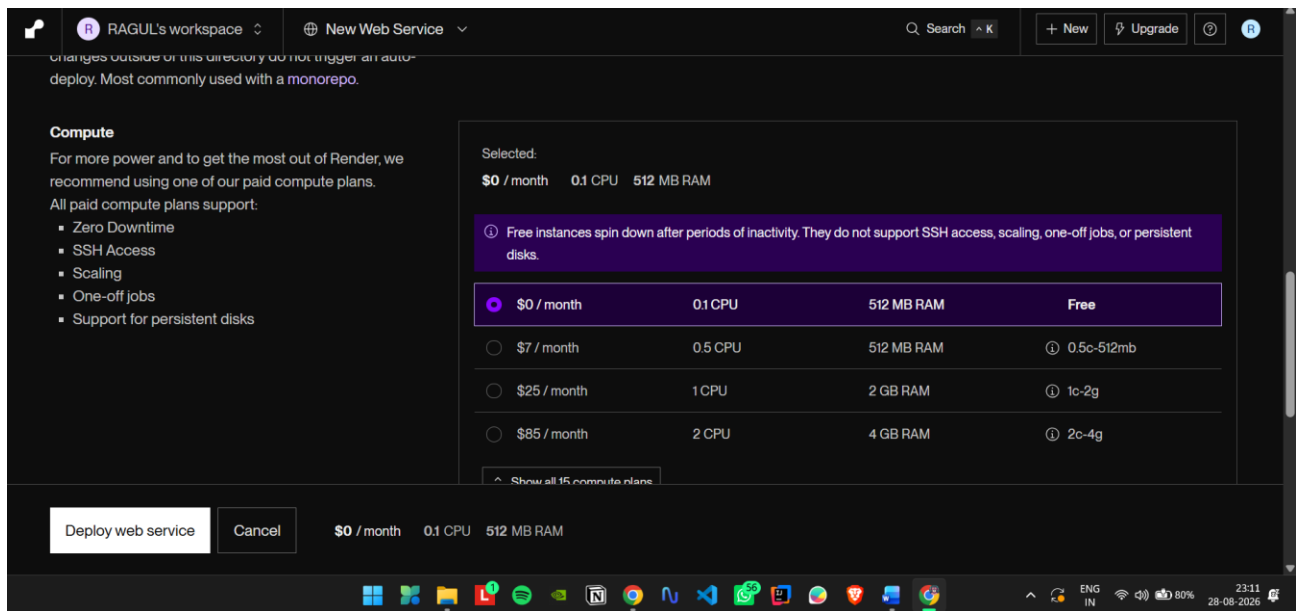
Step 10

Select the **RenderHelloApp** repository from GitHub.

Step 11

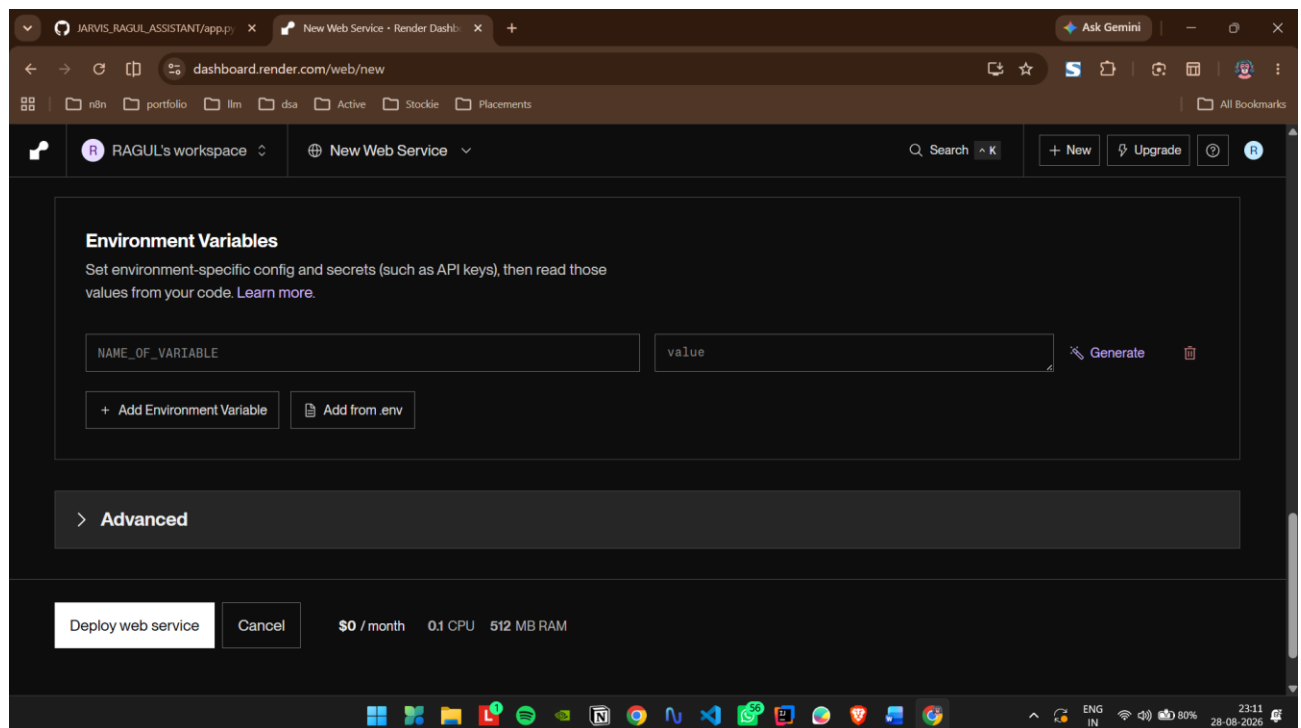
Enter the deployment details:

- **Language:** Python
- **Branch:** main
- **Root Directory:** Leave empty
- **Build Command:** pip install -r requirements.txt
- **Start Command:** gunicorn app:app
- **Instance Type:** Free (if available)



Step 12

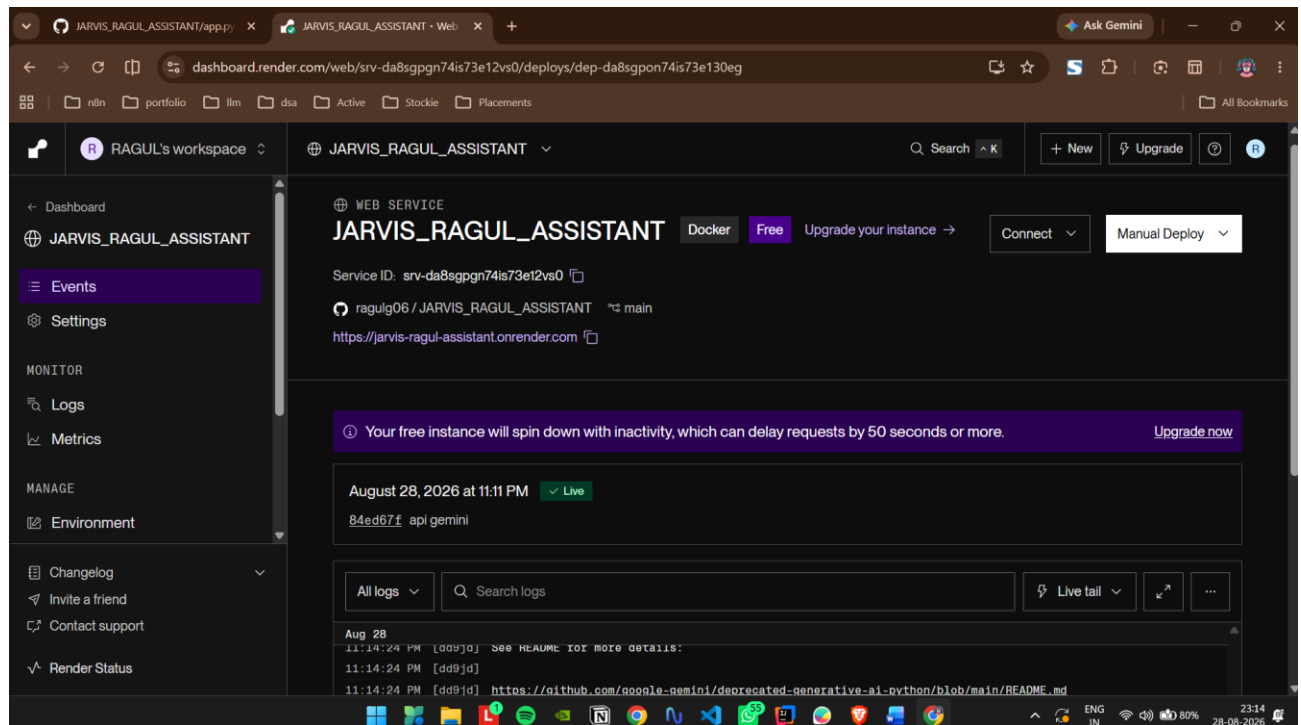
Click Deploy Web Service and wait for the deployment to complete.



Step 13

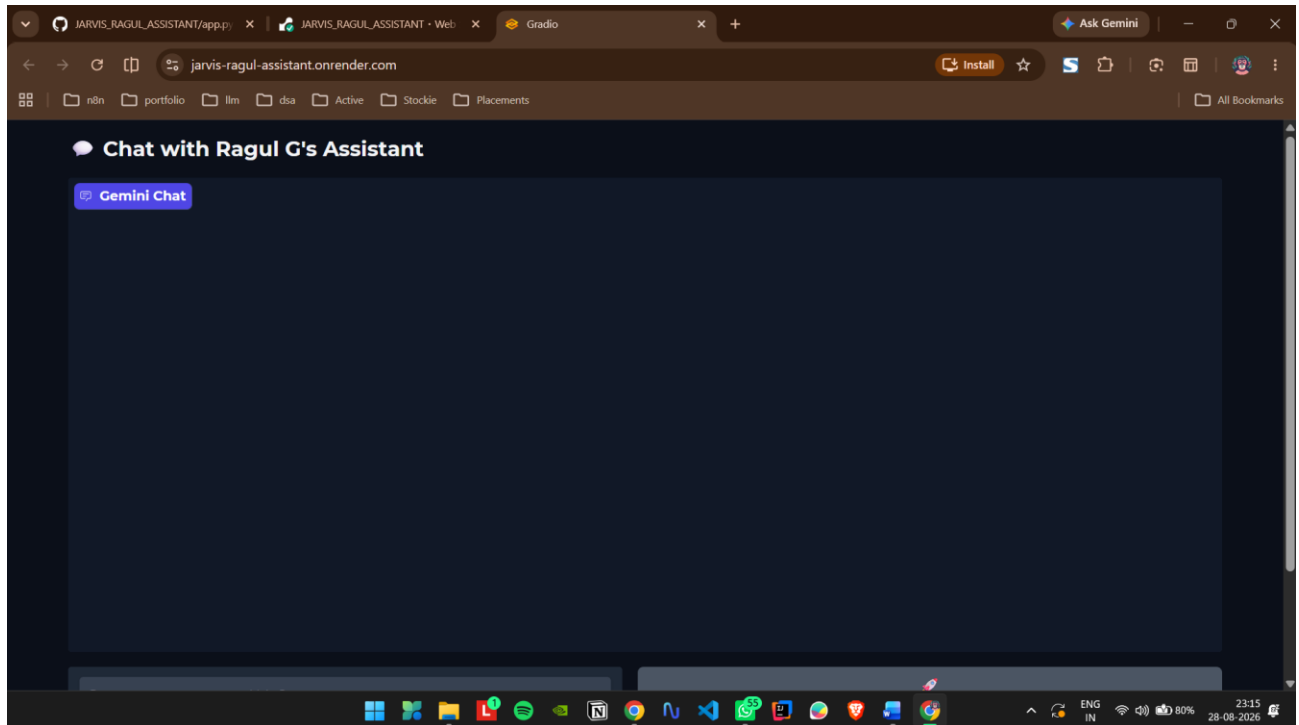
After deployment, open the URL generated by Render and check whether the home page is

displayed correctly.



Step 14

Add **/about** at the end of the application URL to verify that the second route is also working properly.



CONCLUSION:

This experiment demonstrated how a Flask application can be deployed using Render, a Platform as a Service. I learned that PaaS simplifies application deployment by handling the server setup and hosting, allowing developers to publish applications quickly without managing infrastructure.